

Deep Learning Avancé & Applications

Amélioration des Performances
et Généralisation

Master Bac+5 – Data Analytics & Big Data
Mouaad AGOURRAM

2025–2026

Golden Collar

Diagnostic : Overfitting vs Underfitting

Dropout avancé

Early Stopping

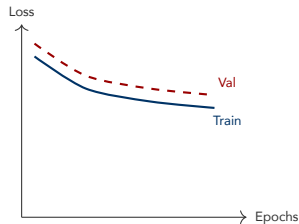
Learning Rate Scheduling

Analyse critique des performances

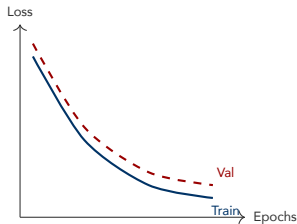
Synthèse

Diagnostic : Overfitting vs Underfitting

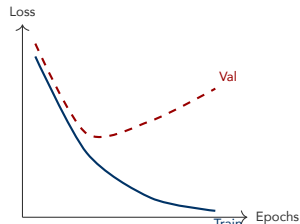
Identifier le problème



Underfitting



Bon ajustement



Overfitting

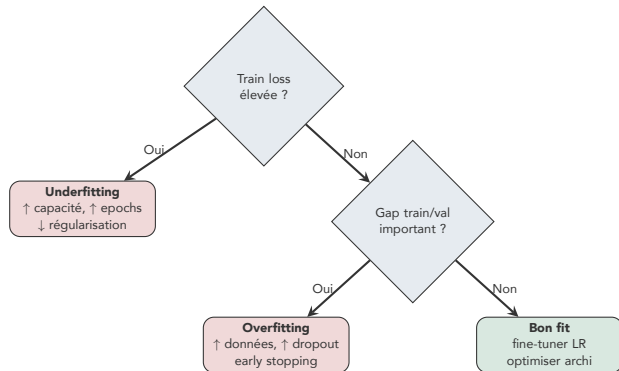
Underfitting

Modèle trop simple $\Rightarrow$ augmenter la capacité, entraîner plus longtemps

Overfitting

Modèle mémorise le train $\Rightarrow$ régulariser, plus de données

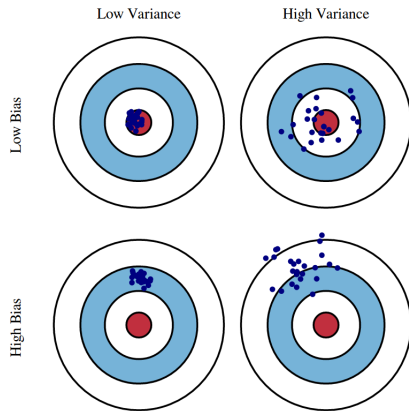
Arbre de décision du diagnostic



Conseil métier

Première action systématique avant toute optimisation : tracer les courbes train/val. En industrie, 80% des problèmes de performance se diagnostiquent à cette étape.

Biais vs Variance : définitions

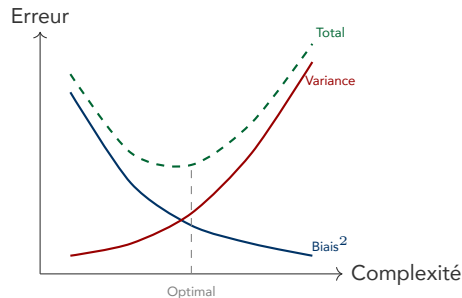


Décomposition de l'erreur

$$\text{Erreur} = \text{Biais}^2 + \text{Variance} + \text{Bruit}$$

- **Biais élevé** : le modèle ne capture pas la complexité $\Rightarrow$ underfitting
- **Variance élevée** : le modèle est trop sensible aux données $\Rightarrow$ overfitting
- **Objectif** : trouver le compromis optimal

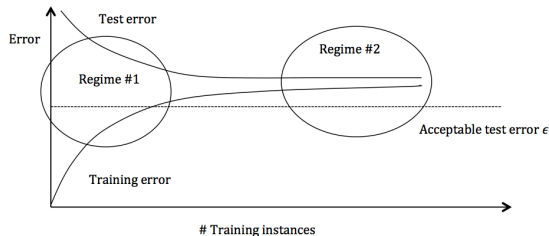
Décomposition exacte pour la perte quadratique (régression) ; en classification, elle s'applique par analogie.



Analyse du problème

Si un modèle sous-performe (erreur d'entraînement ou de test trop élevée), la première étape est de **déterminer la cause racine** :

- **Régime #1** (gauche) : Erreur train $< \epsilon$, mais test $\gg$ train
- **Régime #2** (droite) : Test $\approx$ train, mais les deux $> \epsilon$



Régime #1 : High Variance

Symptômes :

- Erreur train $\ll$ erreur test
- Erreur train $<$ seuil acceptable ϵ
- Erreur test $>$ seuil acceptable ϵ

Remèdes :

- Ajouter plus de données d'entraînement
- Réduire la complexité (*ML classique*) ou **régulariser** (*DL, cf. double descente*)
- Bagging (vu plus tard)

Régime #2 : High Bias

Symptômes :

- Erreur train $>$ seuil acceptable ϵ
- Test $\approx$ train (petit gap)

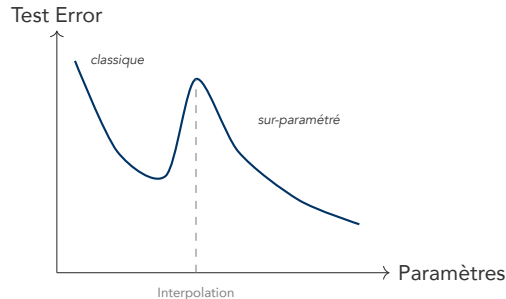
Remèdes :

- Utiliser un modèle plus complexe
- Ajouter de nouvelles features
- Boosting (vu plus tard)

Double descente (Nakkiran et al., 2019)

Contrairement au ML classique, en Deep Learning les modèles très sur-paramétrés peuvent *re-généraliser* :

- Zone classique : la courbe biais-variance standard s'applique
- **Seuil d'interpolation** : pic d'erreur quand $n_{\text{params}} \approx n_{\text{données}}$
- Zone sur-paramétrée : l'erreur *diminue* à nouveau malgré la complexité



Conseil métier

En pratique, cela signifie qu'un **plus gros modèle bien régularisé** bat souvent un petit modèle. Ne réduisez pas la taille du réseau en premier réflexe face à l'overfitting : ajoutez d'abord de la régularisation.

Exemple : diagnostiquer avec les courbes d'apprentissage

```
1 import matplotlib.pyplot as plt
2
3 def plot_diagnostic(history):
4     """Tracer les courbes de diagnostic train vs val."""
5     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
6
7     ax1.plot(history.history['loss'], label='Train')
8     ax1.plot(history.history['val_loss'], label='Validation', linestyle='--')
9     ax1.set_title('Loss'); ax1.legend(); ax1.grid(True, alpha=0.3)
10
11    ax2.plot(history.history['accuracy'], label='Train')
12    ax2.plot(history.history['val_accuracy'], label='Validation', linestyle='--')
13    ax2.set_title('Accuracy'); ax2.legend(); ax2.grid(True, alpha=0.3)
14
15    # Diagnostic automatique
16    gap = history.history['val_loss'][-1] - history.history['loss'][-1]
17    train_final = history.history['loss'][-1]
18    if train_final > 1.0:
19        print("> Diagnostic : UNDERFITTING")
20    elif gap > 0.3:
21        print(f"> Diagnostic : OVERFITTING (gap = {gap:.3f})")
22    else:
23        print(f"> Diagnostic : Bon ajustement (gap = {gap:.3f})")
24
25    plt.tight_layout(); plt.show()
```

Dropout avancé

Dropout classique (Srivastava et al., 2014)

Pendant l'entraînement, chaque neurone est désactivé avec probabilité p :

$$\tilde{h}_i = \begin{cases} 0 & \text{avec prob. } p \\ \frac{h_i}{1-p} & \text{avec prob. } 1 - p \end{cases}$$

Spatial Dropout

- Désactive des *feature maps* entières
- Adapté aux CNN
- Préserve la structure spatiale

Variational Dropout

- Même masque pour toute la séquence
- Adapté aux RNN/LSTM
- Meilleure cohérence temporelle

DropConnect

- Désactive les *poids* (pas les neurones)
- Plus fin que Dropout
- Régularisation plus forte

Pourquoi le Dropout fonctionne ?

Interprétation 1 : Ensemble implicite

- Chaque masque de dropout = un sous-réseau
- n neurones $\Rightarrow 2^n$ sous-réseaux possibles
- Le Dropout entraîne un **ensemble de modèles** partageant les poids
- À l'inférence : moyenne implicite de tous ces sous-réseaux

Interprétation 2 : Anti co-adaptation

- Empêche les neurones de *dépendre excessivement* les uns des autres
- Chaque neurone doit apprendre des features utiles **individuellement**
- Résultat : features plus robustes, plus transférables
- Lien théorique avec la régularisation L2 adaptative

Conseil métier – Quel taux choisir ?

$p = 0.5$ pour les couches **denses**, $p = 0.2$ à 0.3 pour les couches **convolutionnelles**. Ne **jamais** mettre de Dropout juste avant la couche de sortie softmax. Commencer sans Dropout, puis l'ajouter uniquement si overfitting détecté.

Exemple : choisir le bon Dropout selon l'architecture

```
1 from tensorflow.keras import layers
2
3 # -- CNN : utiliser SpatialDropout2D (pas Dropout classique) --
4 x = layers.Conv2D(64, 3, activation='relu')(input_img)
5 x = layers.SpatialDropout2D(0.25)(x) # desactive des feature maps entieres
6
7 # -- Dense : Dropout classique --
8 x = layers.Dense(256, activation='relu')(x)
9 x = layers.Dropout(0.5)(x) # desactive des neurones individuels
10
11 # -- LSTM : Variational Dropout (via recurrent_dropout) --
12 x = layers.LSTM(128,
13     dropout=0.3, # dropout sur les entrees
14     recurrent_dropout=0.3 # meme masque a chaque timestep
15 )(x)
16
17 # -- ATTENTION : ne PAS faire ceci --
18 # x = layers.Dropout(0.5)(x) # apres la derniere couche avant softmax
19 x = layers.Dense(10, activation='softmax')(x) # sortie directe
```

Règle

Dropout classique dans un CNN $\Rightarrow$ inefficace (pixels corrélés). Toujours utiliser SpatialDropout2D.

Le conflit de variance (Li et al., 2019)

- Dropout modifie la variance des activations entre train et inférence
- Placé **juste avant** une BatchNorm, il décale les statistiques apprises par la BN
- Résultat : dégradation en test, parfois sévère

Bonnes pratiques

- Éviter Dropout **juste avant** une BN
- Ordre courant : Conv $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Dropout
- Avec BatchNorm, un Dropout fort est souvent superflu (la BN régularise déjà)
- En pratique : tester avec et sans

Principe (Gal & Ghahramani, 2016)

Utiliser le Dropout **aussi à l'inférence** pour estimer l'incertitude :

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T f(x; \theta, \text{mask}_t) \quad \sigma^2 = \frac{1}{T} \sum_{t=1}^T (f_t - \hat{y})^2$$

- T forward passes avec des masques différents
- La moyenne donne la prédiction, la variance donne l'**incertitude épistémique** (modèle, pas le bruit des données)
- Approximation d'une inférence bayésienne
- **Aucun coût d'entraînement supplémentaire**

Cas d'usage industriels

- **Médecine** : signaler les diagnostics incertains
- **Conduite autonome** : détecter les situations hors distribution
- **Finance** : quantifier le risque
- **Active Learning** : choisir les données à annoter

Exemple : MC Dropout – estimer l'incertitude en 5 lignes

```
1 import numpy as np
2
3 def mc_dropout_predict(model, X, T=50):
4     """T forward passes avec Dropout actif => moyenne + incertitude."""
5     preds = np.array([model(X, training=True).numpy() for _ in range(T)])
6     return preds.mean(axis=0), preds.var(axis=0)
7
8 # Utilisation
9 mean_pred, uncertainty = mc_dropout_predict(model, X_test[:5])
10
11 for i in range(5):
12     classe = np.argmax(mean_pred[i])
13     conf = mean_pred[i].max()
14     incert = uncertainty[i].max()
15     print(f"Sample {i}: classe={classe}, confiance={conf:.2%}, "
16           f"incertitude={incert:.4f}")
```

Conseil métier

En production, définissez un **seuil d'incertitude** au-delà duquel la prédiction est rejetée et renvoyée vers un humain. Ce mécanisme de *reject option* est souvent exigé dans les audits réglementaires (santé, finance).

Early Stopping

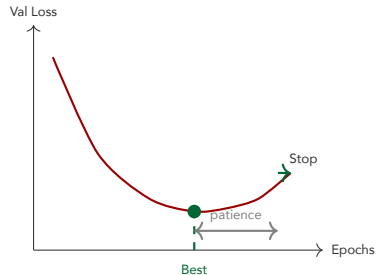
Early Stopping : stratégie optimale

Principe

Arrêter l'entraînement quand la performance sur la validation ne s'améliore plus.

Paramètres clés

- **monitor** : métrique surveillée (val_loss, val_accuracy)
- **patience** : nombre d'epochs sans amélioration tolérées
- **min_delta** : amélioration minimale significative
- **restore_best_weights** : revenir aux meilleurs poids



Patience trop faible (< 5)

- Arrêt sur une simple fluctuation
- Le modèle n'a pas convergé
- Risque d'**underfitting** caché

Patience trop élevée (> 50)

- Gaspillage de GPU (coûteux)
- L'overfitting a le temps de s'installer
- Mais `restore_best_weights` protège

Règles pratiques

Contexte	Patience
CNN classique	10–15
Petit dataset	15–20
Transformer	20–50
Fine-tuning	5–10
Budget GPU serré	5–10

Conseil métier

En entreprise, mettez **toujours** `restore_best_weights=True`. Sans ça, votre modèle final est celui de la *dernière* epoch, pas de la meilleure – erreur surprenamment fréquente en production.

Exemple : Early Stopping + ModelCheckpoint

```
1 from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
2
3 early_stop = EarlyStopping(
4     monitor='val_loss',
5     patience=10,
6     min_delta=1e-4,
7     restore_best_weights=True,
8     verbose=1
9 )
10
11 checkpoint = ModelCheckpoint(
12     'best_model.keras',
13     monitor='val_loss',
14     save_best_only=True
15 )
16
17 history = model.fit(
18     X_train, y_train,
19     validation_data=(X_val, y_val),
20     epochs=200,                # volontairement grand
21     callbacks=[early_stop, checkpoint] # c'est Early Stopping qui decide
22 )
```

Conseil métier

Mettez epochs à un nombre très grand (200, 500...). C'est le callback qui arrêtera. Si vous mettez epochs=50 et patience=10, vous risquez d'arrêter à 50 alors que le modèle s'améliorait encore.

Lien avec la régularisation L2

- En gradient descent, les poids partent de valeurs proches de zéro
- Plus on entraîne longtemps, plus les poids s'éloignent de zéro $\Rightarrow$ modèle plus complexe
- Arrêter tôt $\equiv$ contraindre les poids à rester "petits"
- Équivalent à une **régularisation L2** dans le cas quadratique (approché en général), avec λ implicite lié à la patience

Interactions avec d'autres techniques

- **Early Stopping + Dropout** : complémentaires, à utiliser ensemble
- **Early Stopping + LR Scheduler** : attention, un LR qui décroît peut "tromper" la patience
- **Early Stopping + Data Augmentation** : la convergence est plus lente $\Rightarrow$ augmenter la patience

Conseil métier

En production, ne pas oublier de **logger l'époque d'arrêt**. Si elle varie beaucoup entre les runs (ex: 15 vs 80), c'est un signe d'instabilité de l'entraînement qu'il faut investiguer.

Learning Rate Scheduling

Pourquoi ajuster le Learning Rate ?

Le dilemme du LR constant

- **LR trop grand** : oscille autour du minimum, voire diverge
- **LR trop petit** : convergence extrêmement lente
- Le paysage de la loss évolue au cours de l'entraînement
- Un seul LR **ne peut pas** être optimal du début à la fin

Stratégie optimale

- **Début** : grand LR pour explorer et traverser les plateaux rapidement
- **Milieu** : réduction progressive pour affiner
- **Fin** : petit LR pour converger précisément dans un minimum étroit

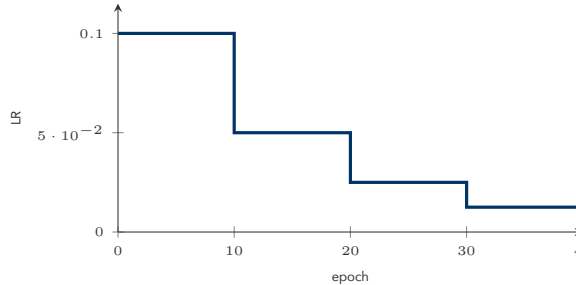
Impact concret (mesuré en pratique)

Un bon scheduling peut améliorer la précision de **1–5%** et réduire le temps d'entraînement de **20–40%**. C'est souvent le levier d'optimisation le plus rentable.

Step Decay

$$\eta_t = \eta_0 \times \gamma^{\lfloor t/s \rfloor}$$

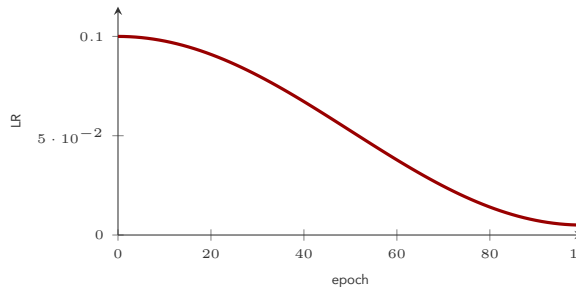
Réduction par facteur γ tous les s epochs.



Cosine Annealing

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\frac{t\pi}{T}))$$

Décroissance douce suivant un cosinus.

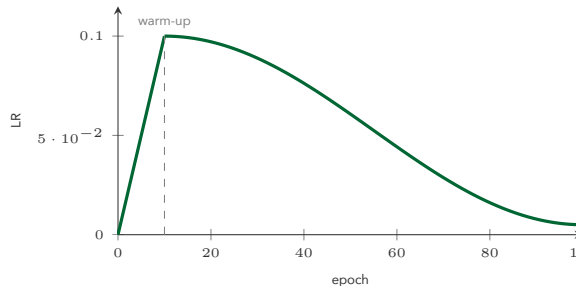


Warm-up + Decay

- Phase 1 : LR croît linéairement (warm-up)
- Phase 2 : décroissance (cosine, linear)
- **Standard pour les Transformers**

Le problème sans warm-up

- Au tout début, les poids sont aléatoires
- Les gradients sont très **bruyants** et instables
- Un grand LR initial $\Rightarrow$ mise à jour violente $\Rightarrow$ paramètres éjectés loin d'un bon minimum
- Particulièrement vrai avec :
 - **Gros batch size** (gradients plus bruités aux premières itérations)
 - **Transformers** (self-attention amplifie les gradients)



ReduceLROnPlateau

- Réduit le LR si la val_loss stagne
- Adaptatif et simple
- Bon choix par défaut

```
1 from tensorflow.keras.callbacks import ReduceLROnPlateau
2
3 reduce_lr = ReduceLROnPlateau(
4     monitor='val_loss',    # surveille la loss de validation
5     factor=0.5,            # divise le learning rate par 2
6     patience=5,            # attend 5 époques sans amélioration
7     min_lr=1e-7,           # learning rate minimum
8     verbose=1              # affiche quand le LR change
9 )
10
11 history = model.fit(
12     X_train, y_train,
13     validation_data=(X_val, y_val),
14     epochs=100,
15     callbacks=[reduce_lr]
16 )
```

Cosine Warm Restarts (SGDR)

- Cosine annealing **redémarré** périodiquement (T_0 , puis cycles croissants)
- Chaque redémarrage relance l'exploration (sort des minima étroits)
- Loshchilov & Hutter (2017)

Snapshot Ensembles

Sauvegarder le modèle **avant chaque redémarrage** : on obtient N modèles en **un seul** entraînement $\Rightarrow$ ensemble quasi gratuit.

OneCycleLR (Smith, 2018)

- **Une seule** oscillation : le LR monte jusqu'à η_{max} puis redescend très bas
- Le momentum varie en sens inverse du LR
- Permet la *super-convergence* : très peu d'epochs
- Idéal quand le budget GPU est serré

Guide de choix : quel scheduler pour quel contexte ?

Scheduler	Quand l'utiliser	Architectures	Hyperparamètres
ReduceLROnPlateau	Prototypage, baseline, incertitude sur le budget d'epochs	Toutes	factor, patience
Step Decay	Reproduire un papier classique	ResNet, VGG	γ , step_size
Cosine Annealing	Entraînement long, compétitions	CNN modernes	T , η_{min}
Warmup + Cosine	Gros batch, pré-entraînement	Transformers, ViT, BERT	warmup_steps
OneCycleLR	Budget d'epochs limité, convergence rapide	EfficientNet, ResNet	max_lr

Conseil métier

Par défaut en entreprise : commencez par ReduceLROnPlateau. C'est adaptatif, sans hyperparamètre critique, et robuste. Passez aux schedules fixes uniquement quand vous maîtrisez le nombre d'epochs nécessaires.

Analyse critique des performances

Au-delà de l'accuracy : les métriques qui comptent

Métriques par classe

- **Precision** = $\frac{TP}{TP+FP}$
parmi tout ce que le modèle a prédit comme positif, combien étaient vraiment positifs ?
- **Recall** = $\frac{TP}{TP+FN}$
parmi tous les vrais positifs existants, combien le modèle en a-t-il détectés ?
- **F1-score** = $2 \cdot \frac{P \cdot R}{P+R}$

Quelle métrique choisir ?

Contexte	Métrique
Équilibré	Accuracy, F1-macro
Déséquilibré	F1-weighted, AUC-PR
Coût des faux positifs	Precision
Coût des faux négatifs	Recall
Ranking, scoring	AUC-ROC
Détection d'objets	mAP
Segmentation	IoU / Dice

Piège de l'accuracy

Dataset avec 95% classe 0, 5% classe 1 :

Prédire toujours 0 $\Rightarrow$ **95% accuracy** mais modèle inutile.

$\Rightarrow$ Toujours regarder **F1, AUC-PR** sur des données déséquilibrées.

Niveau données

- **Oversampling** de la minorité (duplication, SMOTE)
- **Undersampling** de la majorité
- **Attention** : SMOTE est risqué sur images/texte (interpolation peu réaliste)

Niveau apprentissage

- **Class weights** : pondérer la loss (`class_weight` en Keras)
- **Focal Loss** : focaliser sur les exemples difficiles (cf. Séance 1)
- Ajuster le **seuil de décision** (pas toujours 0.5)

Mesurer correctement

Sur données déséquilibrées : F1, AUC-PR, rappel par classe — jamais l'accuracy seule.

1. **Courbes d'apprentissage** : loss et métriques (train vs val)
2. **Matrice de confusion** : identifier les confusions fréquentes entre classes
3. **Courbes ROC / PR** : évaluer la capacité de discrimination
4. **Analyse des erreurs** : examiner les cas mal classifiés *les plus confiants*
5. **Tests statistiques** : significativité des améliorations (bootstrap + t-test)

Pièges courants

- Comparer des modèles sur un seul run (variance !)
- Ignorer le data leakage entre train et validation
- Optimiser sur la validation $\Rightarrow$ surajustement à la validation
- Ne pas utiliser de jeu de test final indépendant

Exemple : matrice de confusion et rapport de classification

```
1 from sklearn.metrics import classification_report, confusion_matrix
2 from sklearn.metrics import ConfusionMatrixDisplay
3 import numpy as np, matplotlib.pyplot as plt
4
5 y_pred = np.argmax(model.predict(X_test), axis=1)
6 y_true = y_test.flatten()
7
8 # Rapport complet en une ligne
9 print(classification_report(y_true, y_pred,
10     target_names=['T-shirt', 'Pantalon', 'Pull', 'Robe', 'Manteau',
11     'Sandale', 'Chemise', 'Basket', 'Sac', 'Botte']))
12
13 # Matrice de confusion visuelle
14 cm = confusion_matrix(y_true, y_pred)
15 ConfusionMatrixDisplay(cm).plot(cmap='Blues')
16 plt.title('Matrice de Confusion')
17 plt.show()
18
19 # Trouver la pire confusion
20 np.fill_diagonal(cm, 0)
21 i, j = np.unravel_index(cm.argmax(), cm.shape)
22 print(f"Confusion la plus frequente : classe {i} -> classe {j} "
23     f"({cm[i,j]} erreurs)")
```

Méthode systématique

1. Trier les erreurs par **confiance décroissante**
2. Les erreurs les plus confiantes = les plus dangereuses
3. Examiner visuellement : bruit ? ambiguïté ? erreur de label ?
4. Chercher des **patterns** dans les erreurs
5. En déduire des actions concrètes

Actions selon les observations

Observation	Action
2 classes confondues systématiquement	Fusionner ou ajouter des données
Erreurs sur images floues	Data augmentation (blur)
Erreurs sur minorité	Oversampling, class weights
Labels erronés	Nettoyage du dataset
Erreurs confiantes	Le modèle a un biais structurel

Conseil métier

Un modèle qui se trompe à **99% de confiance** est plus dangereux qu'un modèle qui se trompe à 55%. En production, les erreurs confiantes passent inaperçues et causent le plus de dégâts.

Calibration : la confiance est-elle fiable ?

Le problème

Un softmax à 0.99 ne signifie pas 99% de chances d'avoir raison. Les réseaux profonds modernes sont souvent **sur-confiants**.

Mesurer : ECE (Expected Calibration Error)

- Regrouper les prédictions par niveau de confiance
- Comparer confiance moyenne vs précision réelle par groupe
- **Reliability diagram** : courbe confiance vs précision (idéal = diagonale)

Corriger

- **Temperature scaling** : diviser les logits par $T > 1$ (post-hoc, simple, efficace)
- **Label smoothing** (cf. Séance 1) : réduit la sur-confiance dès l'entraînement
- **MC Dropout** : une incertitude plutôt qu'un score brut

Pourquoi ça compte

Santé, finance, conduite : on agit sur la **probabilité**, pas seulement sur la classe prédite.

Le problème

Modèle A : 91.2% accuracy

Modèle B : 91.5% accuracy

⇒ B est-il *vraiment* meilleur ?

Ou est-ce juste du **bruit** dû à l'initialisation aléatoire ?

Solutions

- **Plusieurs runs** (5–10 min.) avec seeds différents ⇒ moyenne $\pm$ écart-type
- **Bootstrap** sur le test set ⇒ intervalle de confiance
- **Test de Student apparié** ⇒ p-value
- **Wilcoxon** si distribution non-normale

Ce qu'on attend dans un rapport pro

- Résultats : 91.5% $\pm$ 0.3% (pas juste "91.5%")
- Nombre de runs
- Test statistique + p-value
- **Intervalle de confiance** à 95%

Conseil métier

En industrie, un gain de 0.3% **non significatif** ne justifie pas de redéployer un modèle. Le coût de déploiement, de test, de validation réglementaire est réel. Exigez toujours $p < 0.05$.

Principe

Découper le train en k plis ; entraîner k fois (chaque pli sert une fois de validation).
Performance = moyenne $\pm$ écart-type.

Quand l'utiliser

- **Petits jeux** : estimation plus stable qu'un seul split
- Comparer des modèles avec une variance maîtrisée
- **Stratified k-fold** si classes déséquilibrées

En Deep Learning

- Coûteux (k entraînements) $\Rightarrow$ souvent un seul split train/val sur gros volumes
- **Nested CV** pour régler les hyperparamètres sans biaiser l'estimation
- Le **jeu de test final** reste à part, utilisé une seule fois

Data Leakage : l'ennemi invisible

Définition

Information du **test/validation** qui fuit dans le **train**, donnant des performances artificiellement bonnes.

Sources fréquentes

- Normaliser **avant** de splitter (la moyenne/std inclut le test)
- Données temporelles splittées aléatoirement (futur dans le train)
- Doublons entre train et test
- Feature engineering sur tout le dataset
- Images augmentées d'une même source dans train ET val

Comment détecter

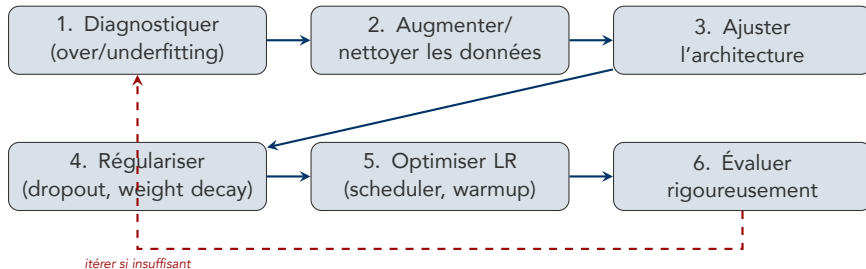
- Performance en val **trop belle** (ex: 99.5% sur un problème dur)
- Val performance $\approx$ train performance dès le début
- Effondrement sur données totalement nouvelles
- Vérifier : `len(set(train_ids) & set(test_ids)) == 0`

Conseil métier

Règle absolue : tout preprocessing (normalisation, feature engineering, tokenization) doit être fit sur le train UNIQUEMENT, puis transform sur val/test.

Synthèse

Synthèse : pipeline d'amélioration



Ce qu'il faut retenir pour la pratique professionnelle

Avant

- ☐ Split sans leakage
- ☐ Normalisation `fit` sur train seul
- ☐ Baseline simple
- ☐ Vérifier l'overfitting sur 1 batch

Pendant

- ☐ Early Stopping + Checkpoint
- ☐ LR Scheduler
- ☐ Dropout / régularisation
- ☐ Monitoring (TensorBoard)

Après

- ☐ Courbes d'apprentissage
- ☐ Matrice de confusion
- ☐ Analyse des erreurs
- ☐ Test statistique
- ☐ Évaluation test final

En pratique

Le jeu de **test** ne sert qu'**une seule fois**, à la toute fin. Si vous l'utilisez pour prendre des décisions de design, il devient un 2^e validation set et votre estimation est biaisée.

Les erreurs qui coûtent cher en production

Erreurs techniques

- Pas de `restore_best_weights` $\Rightarrow$ déployer un modèle sous-optimal
- Data leakage non détecté $\Rightarrow$ performance réelle $\ll$ performance estimée
- Pas de versioning du modèle $\Rightarrow$ impossible de rollback
- Pas de monitoring en production $\Rightarrow$ drift non détecté

Erreurs méthodologiques

- Reporter l'accuracy sur données déséquilibrées
- Comparer avec un seul run
- Optimiser sur val sans test final
- Ignorer les erreurs confiantes
- Ne pas documenter les hyperparamètres

Conseil métier

Un modèle en production qui montre 92% en dev et 85% en prod, c'est un problème de data leakage ou de distribution shift. **Avant de chercher un meilleur modèle, vérifiez vos données.**

TD : Diagnostic et correction de l'overfitting

- Réseau volontairement en overfitting sur Fashion-MNIST
- Appliquer progressivement : Dropout, Early Stopping, Data Augmentation
- Tracer et comparer les courbes d'apprentissage

TP : Impact du Learning Rate Scheduling

- Entraîner un CNN sur CIFAR-10 avec différents schedulers
- Comparer : constant, step decay, cosine, warmup+cosine
- Combiner avec ReduceLROnPlateau
- Produire un rapport d'analyse avec visualisations

Questions ?

Séance suivante : *Architectures CNN avancées*